

ENTERPRISE TECHNICAL ARCHITECTURE & PROJECT DOCUMENTATION

System: Online Complaint Registration & Management System (OCRMS)

1. Document Control

Attribute	Details
Document Title	Enterprise Architecture & System Design Specification
Project Name	Online Complaint Registration & Management System (OCRMS)
Document Version	1.0.0 (Baseline Release)

2. Executive Summary

The **Online Complaint Registration & Management System (OCRMS)** is a centralized, role-based ticketing platform engineered to streamline issue reporting, track resolution lifecycles, and facilitate direct communication between stakeholders. Designed utilizing a modern three-tier MERN (MongoDB, Express.js, React.js, Node.js) architecture, the system provides a robust solution for organizational complaint management.

The platform addresses the critical business need for operational transparency by replacing disparate reporting channels with a unified, auditable digital workflow. Key capabilities include strict **Role-Based Access Control (RBAC)**, multi-category ticket classification, real-time visual timeline tracking, secure file attachments, and data-driven administrative analytics.

The architecture emphasizes security at the gateway layer (JWT authentication, rate limiting, helmet headers) and ensures data integrity via strict relational referencing and embedded sub-documents in MongoDB. The current iteration serves as a production-ready baseline, designed with decoupling in mind to facilitate future horizontal scaling and enterprise deployment.

3. Project Background

Modern organizations face significant operational overhead when managing internal or customer-facing issues without a centralized system. Traditional approaches such as email threads, paper forms, or fragmented spreadsheets result in opaque resolution processes, lost attachments, misaligned agent workloads, and a lack of metric-driven oversight.

The Engineering Challenge:

The objective was to architect a unified digital ecosystem that guarantees complaint immutability (audit trails), enforces strict access boundaries (RBAC), securely handles user-generated file uploads, and provides real-time visibility into the status of active tickets. The solution required decoupling the client presentation logic from the backend business rules while maintaining high performance during complex aggregate queries for administrative dashboards.

4. Problem Statement

Organizations currently lack a streamlined, traceable, and secure mechanism for complaint lifecycle management. Specific pain points include:

- **Opaque Workflows:** Users have no visibility into the status of their submitted issues.
- **Communication Silos:** Agents and users lack a context-aware channel for direct communication, leading to delayed resolutions.
- **Resource Misallocation:** Administrators cannot accurately measure agent workload, ticket volume by department, or SLA compliance.
- **Security Vulnerabilities:** Existing manual systems fail to enforce authorization boundaries, risking unauthorized access to sensitive complaint data.

5. Objectives

Primary Objectives

- Develop a secure, centralized portal for complaint registration and tracking.
- Implement a deterministic state machine for ticket lifecycles (**Pending** -> **In Progress** -> **Resolved** -> **Closed/Rejected**).
- Provide real-time, context-aware messaging threads tied to specific tickets.

Secondary Objectives

- Deliver automated and manual ticket assignment mechanisms based on category and workload.
- Enable post-resolution satisfaction metrics (1–5 star ratings).
- Generate administrative analytics and exportable reporting (CSV/PDF).

Security Objectives

- Enforce strict API access controls using short-lived stateless **JWTs**.
- Protect against brute-force and DoS attacks via targeted rate limiting (100 req/15min).
- Sanitize and validate all multi-part form data to prevent malicious file execution.

6. Scope

In Scope (Implemented)

- Full MERN stack implementation with **React Single Page Application (SPA)**.
- Three-tier **Role-Based Access Control (USER, AGENT, ADMIN)**.
- Ticket lifecycle management with immutable audit trails.
- In-ticket bi-directional messaging.
- Local file system attachment handling with MIME/Size validation (5MB).
- Administrative dashboard with aggregation-based KPI metrics.

Out of Scope / Future Expansion (Proposed)

- Migration of local file uploads to scalable cloud object storage (e.g., AWS S3).
- Real-time bidirectional communication via WebSockets (Socket.io) for messaging.
- AI-driven automated ticket categorization and sentiment analysis.
- Microservices decomposition of the Express monolith.

7. Stakeholders and Users

- **USER (Citizen/Employee/Client):** Submits tickets, monitors visual steppers, provides feedback, and communicates with assigned agents.
- **AGENT (Support Specialist):** Consumes the assigned ticket queue, transitions ticket statuses, adds official remarks, and resolves issues.
- **ADMIN (System Supervisor):** Manages the entire organizational queue, reassigns tickets, monitors system analytics, and exports audit logs.

- **Security/DevOps Engineers (Internal):** Responsible for infrastructure deployment, secrets management, and log monitoring.

8. Functional Requirements

ID	Requirement	Description	Priority	Component
FR-001	Authentication	System must authenticate users via email/password and issue JWTs.	Critical ▾	Auth Controller
FR-002	RBAC	System must strictly enforce USER, AGENT, and ADMIN access scopes.	Critical ▾	Security Middleware
FR-003	Ticket Creation	Users must be able to submit complaints with category, priority, and attachments.	High ▾	Complaint Controller
FR-004	Audit Trail	Status changes must auto-generate a timestamped, immutable audit log.	High ▾	Complaint Model
FR-005	Messaging	Users and Agents must be able to exchange messages within a specific ticket context.	High ▾	Message Controller
FR-006	Admin Analytics	Admins must view aggregated KPIs (Total, Resolution Rate, Avg Time).	Medium ▾	Admin Controller

9. Non-Functional Requirements

ID	Requirement	Description
NFR-001	Security	Passwords must be hashed using <code>bcryptjs</code> (10 rounds). APIs must use Helmet and CORS .
NFR-002	Performance	API response times should remain under 200ms for standard CRUD operations.
NFR-003	Data Integrity	Mongoose schemas must enforce referential integrity using <code>ObjectId</code> pointers.
NFR-004	Scalability	The stateless JWT architecture must allow the Express layer to scale horizontally.
NFR-005	File Safety	File uploads must be strictly limited to 5MB and validated against safe MIME types (Images/PDFs).

10. Technology Stack

Technology	Purpose	Why Used	Layer
React.js + Vite	SPA Framework	Vite provides ultra-fast HMR and optimized production builds. React enables component-driven UI architecture.	Presentation ▾
Material-UI (MUI)	UI Design System	Ensures enterprise-grade, accessible, and responsive user interfaces with pre-built complex components (e.g., Steppers, Data Grids).	Presentation ▾
Express.js / Node.js	REST API Runtime	Non-blocking I/O model is ideal for I/O heavy operations (database queries, file uploads).	Logic / API ▾

Technology	Purpose		Why Used	Layer
MongoDB / Mongoose	NoSQL Database		Flexible document schema accommodates heterogeneous complaint data. Mongoose provides necessary validation and lifecycle hooks.	Persistence ▾
JSON Web Tokens	Auth Mechanism		Enables stateless, scalable authentication without requiring server-side session memory.	Security ▾
Multer	Multipart Parser	Form	Safely handles incoming binary file streams before they reach business logic.	Security/Middl... ▾
bcryptjs	Cryptography		Mitigates rainbow table attacks via computationally expensive salted password hashing (10 rounds).	Security ▾

11. System Architecture

11.1 High-Level Architecture

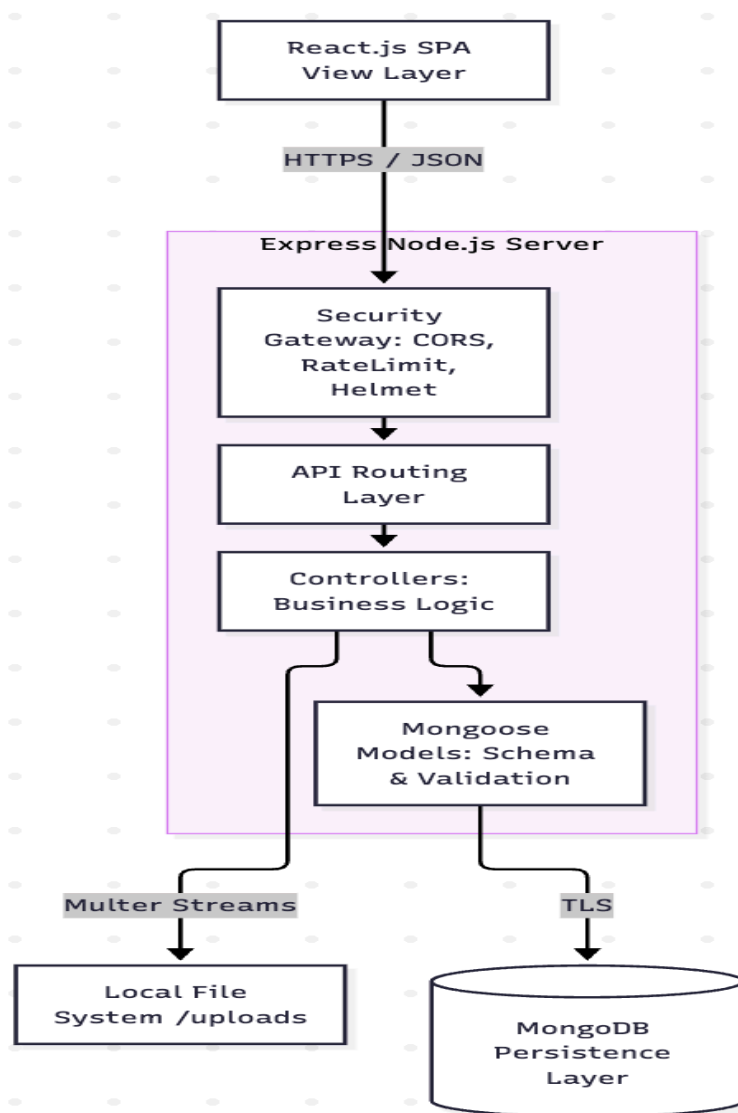
The system employs a strict **Model-View-Controller (MVC)** paradigm within a Client-Server architecture. The React SPA communicates with the Express REST API via HTTPS, passing Bearer tokens for state validation.

- Client (React.js SPA View Layer) communicates via HTTPS/JSON to the Node.js Server Gateway.
- The Gateway handles CORS, RateLimiting, and Helmet security before passing to the API Routing Layer.
- The Router passes requests to Controllers for business logic.
- Controllers interact with Mongoose Models (Schema & Validation).
- Models connect to MongoDB for persistence.
- Controllers manage file streams and save to the Local File System.

11.2 Detailed Component Architecture

The architecture deeply couples middleware with routing to ensure fail-safe authorization boundaries.

1. User POSTs to `/api/complaints` with **FormData** + **JWT**.
2. Middleware validates **JWT Signature**, checks **User Role**, and validates **Multer 5MB limits**.
3. Controllers execute DB operations.
4. Mongoose executes pre-save hooks and returns saved document.
5. **201 Created** payload returned to the client.



12. Database Architecture (ER & Design)

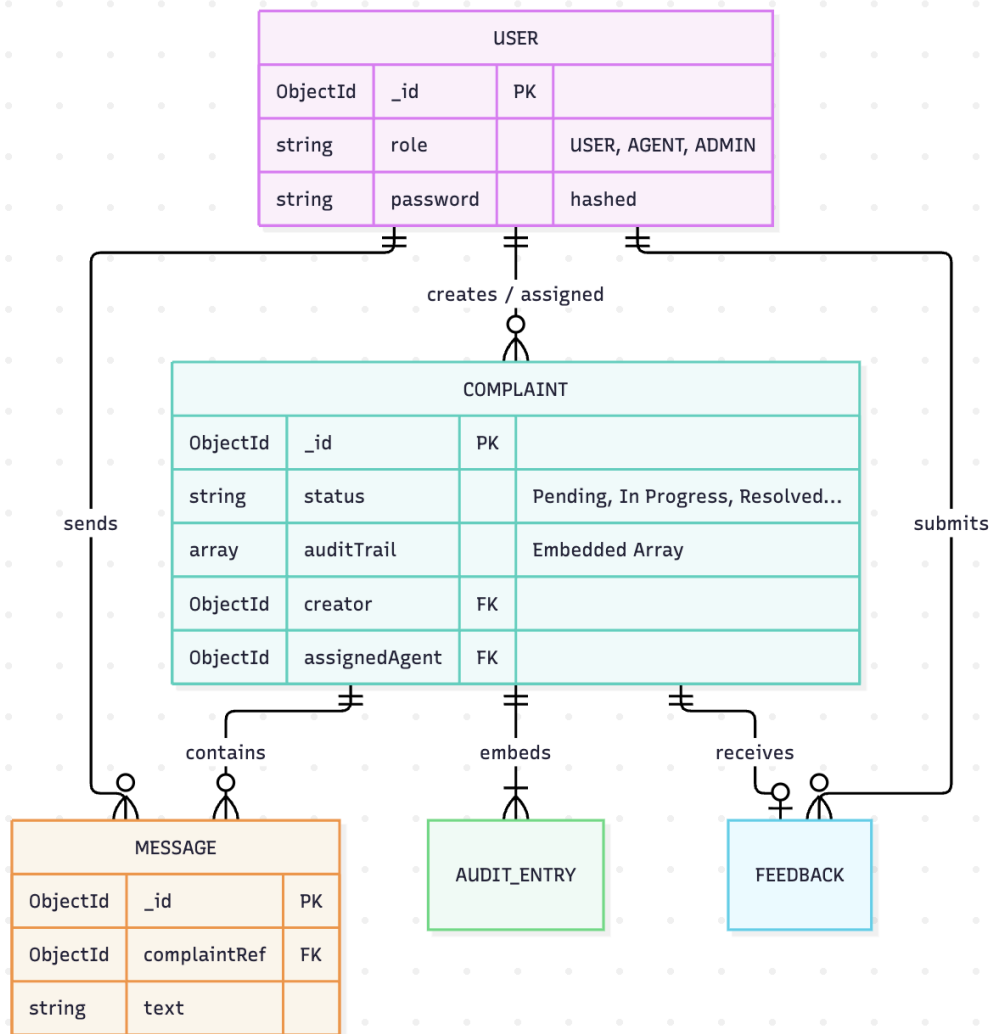
The database schema is designed to balance the benefits of NoSQL document embedding with relational referencing to prevent data duplication and unbounded document growth.

- USER: `_id` (PK), role, password (hashed)
- COMPLAINT: `_id` (PK), status, auditTrail (Embedded Array), creator (FK), assignedAgent (FK)
- MESSAGE: `_id` (PK), complaintRef (FK), text

Relationships: User creates/is assigned to Complaints. User sends Messages. Complaints contain Messages and embed Audit Entries.

12.1 Schema Design Decisions

- **Embedded Audit Trail:** The `auditTrail` is embedded within the `Complaint` document. *Why?* Because audit logs are tightly coupled to the ticket and are almost always read concurrently with the ticket. Embedding avoids an expensive join.
- **Referenced Messages:** `Message` documents are referenced via `complaintRef` rather than embedded. *Why?* To prevent the `Complaint` document from exceeding MongoDB's 16MB document limit in cases of extensive communication threads.



13. Module-by-Module Documentation

13.1 Authentication & Profile Module (**authController**)

- **Purpose:** Manages identity verification, session state, and cryptographic asset generation.
- **Processing:** Intercepts login requests, queries MongoDB by email, uses `bcryptjs.compare()` against the stored hash. Upon success, signs a JWT with user ID and Role.
- **Security:** Implements time-limited crypto-tokens for password resets sent via SMTP to prevent unauthorized account recovery.

13.2 Ticket Lifecycle Module (complaintController)

- **Purpose:** The core engine of the system. Manages ticket creation, state transitions, and file persistence.
- **Processing:** Validates multipart form data. On status updates (Pending \$→\$ In Progress), it automatically appends a timestamped entry to the auditTrail array including the acting agent's ID and mandatory remarks.
- **Error Handling:** Rolls back file system saves if the database transaction fails to prevent orphaned files in /uploads.

13.3 Analytics & Control Module (adminController)

- **Purpose:** Provides executive oversight and workload distribution.
- **Processing:** Utilizes advanced MongoDB Aggregation Pipelines to calculate average resolution times and categorical breakdowns without loading raw datasets into Node.js memory.

14. API Documentation (Key Endpoints)

Method	Endpoint	Auth Level	Purpose	Input / Body
POST ▾	/api/auth/login	Public ▾	Authenticate and retrieve JWT	{ email, password } ▾
POST ▾	/api/complaints	USER ▾	Submit a new complaint ticket	FormData (text + files) ▾
GET ▾	/api/complaints/my-complaints	USER ▾	Retrieve user's ticket list	N/A ▾
PUT ▾	/api/complaints/:id/status	AGENT/AD... ▾	Advance ticket state machine	{ status, remark } ▾
GET ▾	/api/admin/analytics	ADMIN ▾	Retrieve aggregated system KPIs	N/A ▾
POST ▾	/api/messages/:complaintId	All Roles ▾	Post message to ticket thread	{ text } ▾

15. Security Architecture

Security is treated as a first-class engineering concern, mapped against OWASP principles.

15.1 Implemented Security Controls

- **Network Security:** `Helmet.js` strips identifying server headers and enforces strict HTTP security headers.
- **DDoS / Brute Force Mitigation:** Express Rate Limiter restricts requests to 100 per 15 minutes per IP.
- **Authentication & Authorization:** Zero-trust approach on API routes. The JWT is verified on every protected request.
- **Data Validation:** `Multer` is configured to reject files larger than 5MB and files outside explicit MIME types, mitigating Remote Code Execution (RCE).
- **Data at Rest:** Passwords are never stored in plaintext; salted and hashed via `bcryptjs`.

15.2 Threat Model (STRIDE)

Threat	Attack Vector	Mitigation (Implemented)
Spoofting	Forging identities to view other's tickets.	JWT signature validation using a highly entropic server-side secret.
Tampering	Modifying ticket status maliciously.	Backend enforces RBAC. Only AGENT/ADMIN can trigger the state machine API.
Repudiation	"I didn't close that ticket."	Immutable <code>auditTrail</code> tracks the exact <code>userId</code> and timestamp of every state change.
Denial of Service	Spamming the file upload endpoint.	Rate Limiting + Multer 5MB hard limit prevents disk exhaustion.

16. Data Flow: Ticket Creation & Processing

1. **Input:** User fills React form. Axios compiles a `FormData` object containing JSON strings and binary file Blobs.
2. **Gateway:** Request hits Express. CORS accepts origin. Rate Limiter checks IP quota.
3. **Authentication:** Auth middleware extracts Bearer JWT, verifies signature, decodes payload, attaches `req.user`.

4. **Middleware:** Multer intercepts stream, validates MIME/Size, saves to disk `/uploads`, attaches `req.files`.
5. **Processing:** Controller maps `req.body` and `req.files` to Mongoose schema.
6. **Persistence:** Mongoose executes `pre('save')` hooks, validates enumerators, and saves to MongoDB.
7. **Response:** 201 Created payload returned to React, which updates the local Context/Redux state and transitions UI.

17. Error Handling & Observability

Implemented Mechanisms

- **Global Error Handler:** Express utilizes a centralized error-handling middleware. It intercepts Mongoose `ValidationError`, `CastError`, and JSON parsing errors, returning standardized JSON error envelopes.
- **HTTP Status Codes:** Strict adherence to REST semantics (400, 401, 403, 404, 500).
- **File Cleanup:** If database insertion fails during ticket creation, a synchronous `fs.unlink()` cleanup function executes to remove the dynamically uploaded file.

18. Performance and Scalability

Architectural Scalability

- **Stateless API:** By relying on JWTs rather than server-side sessions, the Node.js backend is entirely stateless. This allows horizontal scaling behind a Load Balancer without requiring sticky sessions.
- **Database Indexing:** MongoDB is indexed on `Complaint.status` to optimize Agent Queue retrieval, and indexed on `Complaint.creator` to ensure instant loading of the User Dashboard.

Current Limitations (To be addressed in Production)

- **Stateful File Uploads:** Files are currently saved to the local file system. If scaled horizontally across multiple Node.js instances, files uploaded to Instance A will result in 404s when requested by a client connected to Instance B.
 - *Proposed Solution:* Integrate AWS S3 SDK in the controller layer to offload object storage.

19. Development Challenges & Design Decisions

19.1 Architecture Decision Record (ADR): Ticket Status Mutability

- **Decision:** Implement a strict one-way state machine for ticket statuses with an append-only audit log.
- **Alternatives Considered:** Allowing agents to arbitrarily edit the `status` field.
- **Reasoning:** Compliance and accountability require a forensic trail.
- **Trade-offs:** Increases database storage requirements slightly per ticket, and complicates the UI stepper logic.

19.2 Architecture Decision Record (ADR): Direct Messaging Storage

- **Decision:** Store messages in a separate `Messages` collection linked by `complaintRef`.
- **Alternatives Considered:** Embedding messages as an array inside the `Complaint` document.
- **Reasoning:** MongoDB has a hard limit of 16MB per document. Direct messaging threads are unbounded and could exceed limits. Separating them ensures infinite scalability.

20. Recommended Production Deployment Architecture

While currently designed for local development, the system is engineered to transition to the following enterprise topology:

- A **Web Application Firewall (WAF)** passes traffic to a Load Balancer.
- The **Load Balancer** directs traffic to multiple Node.js instances in a private VPC.
- The **Node.js instances** connect to a MongoDB Atlas Cluster and use the AWS S3 API for file attachments.
- Environment configurations inject credentials securely via **CI/CD pipelines**.

21. Future Roadmap

- **Phase 1 — Stabilization (Current):** MVP complete. File system uploads, monolithic API, polling-based UI updates.
- **Phase 2 — Real-Time Upgrades:** Replace HTTP polling for messaging with WebSocket (Socket.io) integration for instantaneous bi-directional chat without page refreshes.
- **Phase 3 — Cloud Native:** Refactor Multer middleware to use `multer-s3`, migrating attachment storage to cloud object storage.

- **Phase 4 — Advanced Intelligence (Proposed):** Implement an AI-driven categorization engine (using an LLM API) to automatically route incoming tickets to the correct department based on NLP analysis.

22. Conclusion

The **Online Complaint Registration & Management System** demonstrates a robust, enterprise-aware approach to resolving operational workflow challenges. By enforcing strict architectural boundaries between data presentation, business logic, and database persistence, the system achieves a highly secure and auditable operational environment. The integration of role-based authorization, defensive security middleware, and a strictly referenced MongoDB schema highlights significant technical depth, providing a reliable foundation capable of scaling into a mature organizational platform.